Priority queues and binary heaps

COMP 182 Lecture 11

Operations, implementation choices, and time complexity

n = number of stored items
Bounds assume constant-time comparisons

The priority queue ADT

Each removal returns an item with the highest priority under the chosen ordering.

Example: an emergency department treats more urgent cases first.

A max priority queue returns the greatest key.

A min priority queue returns the smallest key.

Equal-priority items need an explicit rule if FIFO order matters.

The ADT defines behavior. The implementation determines cost.

A consistent Java interface

This course interface returns null when peek() or remove() finds an empty queue.

```
public interface PriorityQueueInterface<T
        extends Comparable<? super T>> {
    boolean add(T newEntry);
    T remove();
    T peek();
    boolean isEmpty();
    int size();
    void clear();
}
```

Use T consistently in both the type parameter and method signatures.

Priority queue implementations

Worst-case time for n items. The next slides explain each representation.

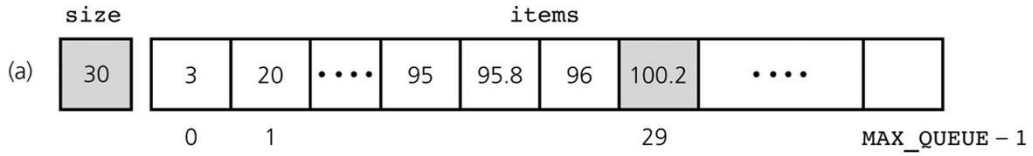
Representation	Insert	Peek / remove extreme
Unsorted array	O(1) amortized	O(n) / O(n)
Sorted array	O(n)	O(1) / O(1)
Sorted linked list	O(n)	O(1) / O(1)
Balanced BST	O(log n)	O(log n) / O(log n)
Binary heap	O(log n)*	O(1) / O(log n)

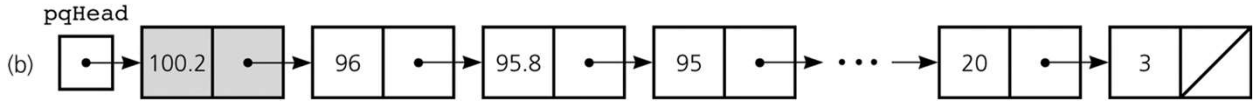
*Heap repair is O(log n). A resizing insertion can take O(n).

Arrays remove at the end. Lists remove at the head. BST has no cached extreme.

Max priority queues with sorted storage

Both representations place the maximum where removal needs no search.





Sorted array: binary search O(log n), then shifting O(n).

Sorted linked list: search O(n), then relink O(1).

Insert O(n) Peek O(1) Remove maximum O(1)

Original diagram: © 2011 Pearson Addison-Wesley

Min priority queues with sorted storage

Reverse the ordering so the minimum occupies the removal position.

Representation	Ordering	Minimum location
Sorted array	Descending	Last occupied index
Sorted linked list	Ascending	Head

An array sorted in ascending order needs O(n) shifting to remove index 0.

With the ordering above: insert O(n), peek O(1), remove minimum O(1)

A binary search tree implementation

The maximum is the rightmost node. Its location depends on tree height h.

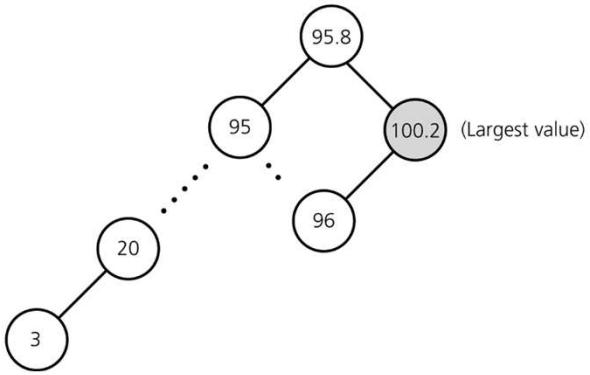
Insert: O(h)

Find and remove maximum: O(h)

Balanced BST: h = O(log n)

Unbalanced BST: h can be n − 1

(c)



A plain BST can degrade to O(n) per operation.

Original diagram: © 2011 Pearson Addison-Wesley

The two heap properties

A binary heap combines a complete tree shape with a local ordering rule.

Shape: every level is full except possibly the last.

The last level fills from left to right.

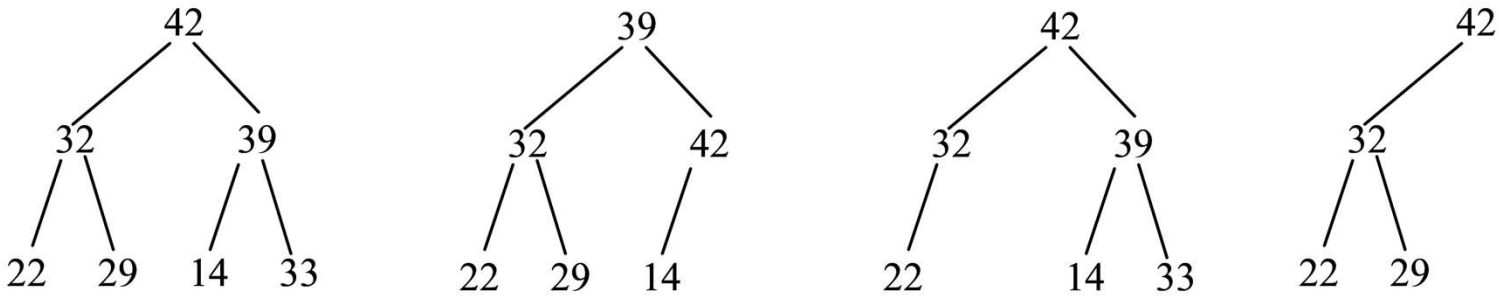
Max-heap order: each parent key is at least each child key.

Every subtree satisfies the same ordering rule.

A heap supports fast access to one extreme. Arbitrary search is O(n).

Complete trees and heap order

Completeness constrains the shape. Heap order constrains the keys.



(a) Complete and a max-heap. (b) Complete, but 39 < 42 violates heap order.

(c) and (d) are incomplete because the final level has gaps.

For $n \geq 1$, a complete binary tree has height $\lfloor \log_2 n \rfloor$.

Original diagram: © 2011 Pearson Addison-Wesley

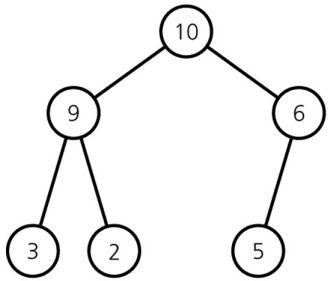
Max-heaps and min-heaps

Max-heap: parent ≥ children.

The root contains a maximum.

Min-heap: parent ≤ children.

The root contains a minimum.



0	10
1	9
2	6
3	3
4	2
5	5

Peek at the root: O(1) in either heap.

Original diagram: © 2011 Pearson Addison-Wesley

Heap operation costs

Array-backed binary heap with no shrinking during removal.

Operation	Time	Reason
Empty constructor	O(1)	No items to process
peek / size / isEmpty	O(1)	Read root or count
Insert	O(log n)*	Sift up one path
Remove root	O(log n)	Sift down one path
clear	O(n)	Null occupied references

*Repair bound. Dynamic array growth can make one insertion O(n).
With geometric growth, insertion is O(log n) amortized.

An array representation

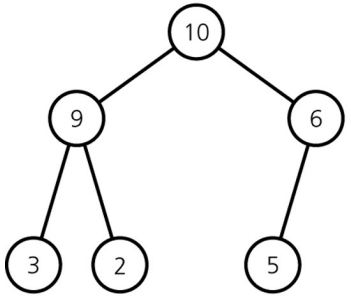
Level-order storage keeps the complete tree compact. Indices start at 0.

Root: index 0

Left child of i: 2i + 1

Right child of i: 2i + 2

Parent of i > 0: [(i − 1) / 2]



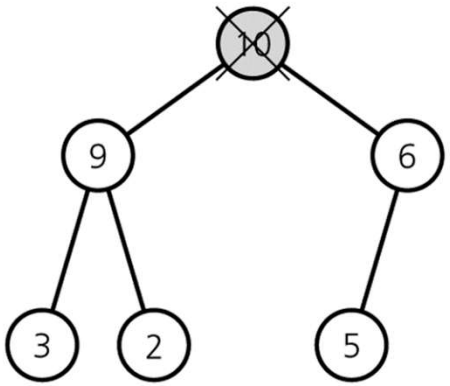
0	10
1	9
2	6
3	3
4	2
5	5

Index calculations and random access take O(1). Storage is O(n).

Original diagram: © 2011 Pearson Addison-Wesley

Remove the root: save the result

Save the maximum before changing the heap.



(a)

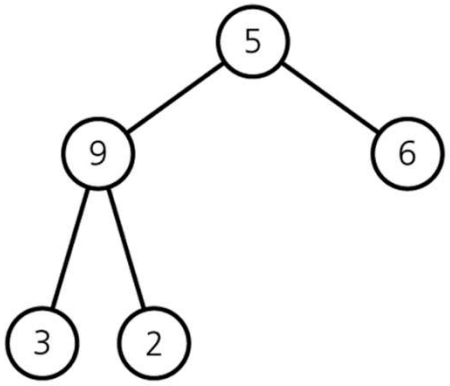
0	~~10~~
1	9
2	6
3	3
4	2
5	5

Saving the root takes O(1). Repair comes after replacing it.

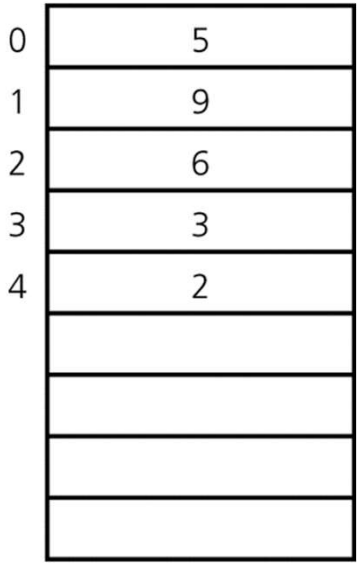
Original diagram: © 2011 Pearson Addison-Wesley

Remove the root: replace it with the last item

Move the final array entry to index 0, then remove the final entry.



(b)



Replacement and removal at the array end take O(1).

Original diagram: © 2011 Pearson Addison-Wesley

The intermediate array

Only the root may violate heap order. The two child subtrees remain heaps.

Index	0	1	2	3	4
Value	5	9	6	3	2

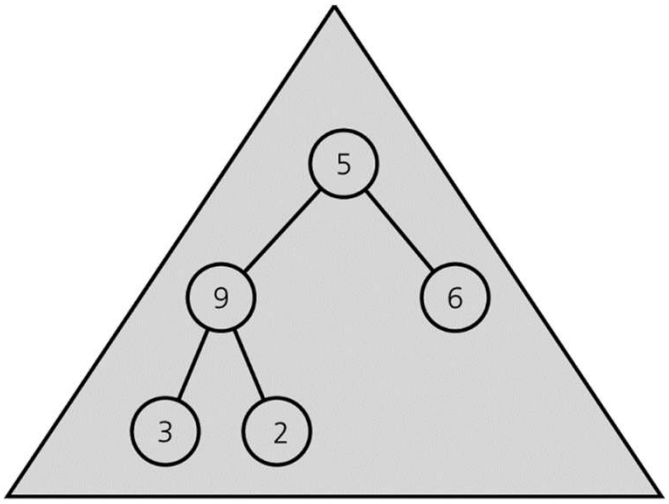
The value 5 is smaller than its child 9.

Compare both children and choose the larger child for a max-heap.

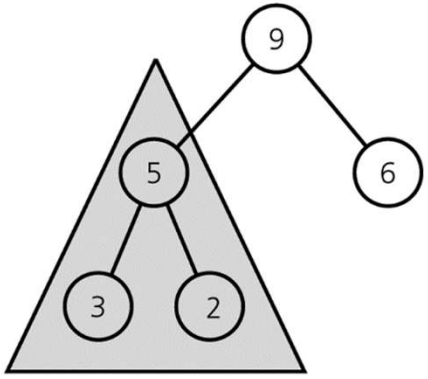
Each level requires O(1) comparisons and at most one swap.

Sift down restores heap order

Swap with the larger child until the parent is large enough or reaches a leaf.



First semiheap passed to `heapRebuild`



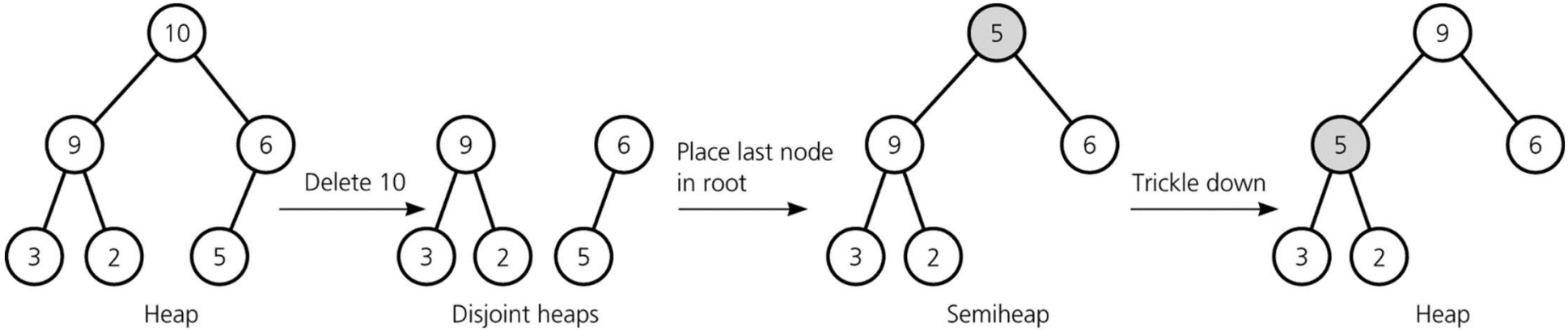
Second semiheap passed to `heapRebuild`

At most $\lfloor \log_2 n \rfloor$ levels. Sift down takes O(log n) worst-case.

Original diagram: © 2011 Pearson Addison-Wesley

Deletion: the complete sequence

Save root, replace it with the last item, and sift down.



The complete shape stays intact because only the last node disappears.

Remove root: O(log n) worst-case, O(1) best-case.

Original diagram: © 2011 Pearson Addison-Wesley

Sift down: iterative Java

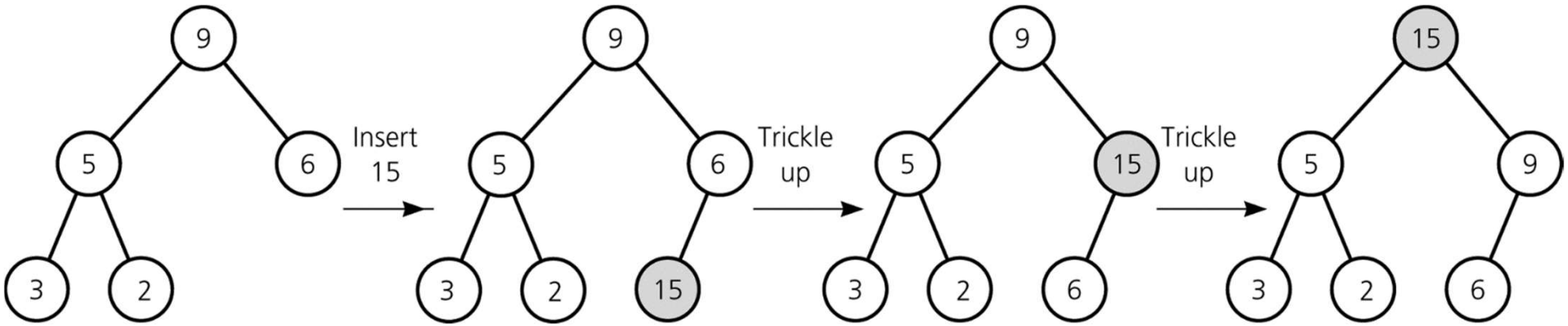
Both child subtrees must already satisfy max-heap order.

```
private void heapRebuild(int root) {
    int n = list.size();
    while (2 * root + 1 < n) {
        int child = 2 * root + 1;
        if (child + 1 < n &&
            list.get(child + 1).compareTo(list.get(child)) > 0)
            child++;
        if (list.get(root).compareTo(list.get(child)) >= 0)
            break;
        Collections.swap(list, root, child);
        root = child;
    }
}
```

Time O(log n). Extra space O(1), versus O(log n) for recursive calls.

Insertion: append, then sift up

Append at the next free position to preserve the complete shape.



Compare the new item with its parent and swap while the item is larger.

Heap repair takes O(log n) worst-case and O(1) best-case.

Original diagram: © 2011 Pearson Addison-Wesley

Sift up: Java implementation

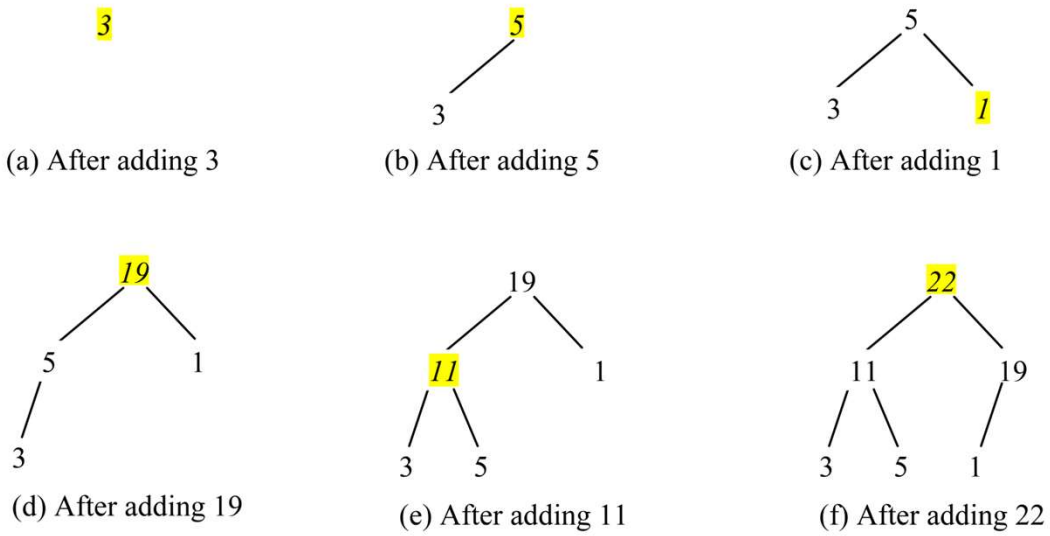
The loop condition stops before trying to move above the root.

```
public void add(T newItem) {
    list.add(newItem);
    int i = list.size() - 1;
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (list.get(i).compareTo(list.get(parent)) <= 0)
            break;
        Collections.swap(list, i, parent);
        i = parent;
    }
}
```

O(log n) amortized with ArrayList. A resizing insertion can take O(n).

Insertion example: 3, 5, 1, 19, 11, 22

Begin with an empty max-heap. Each highlighted value has just been inserted.

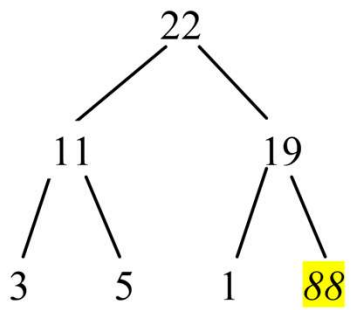


Six insertions here. In general, n successive insertions cost O(n log n).

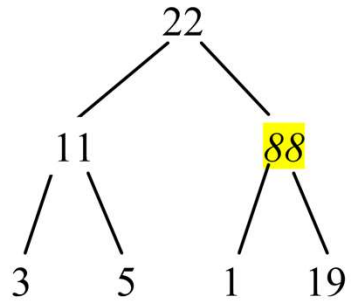
Original diagram: © 2011 Pearson Addison-Wesley

Insertion example: add 88

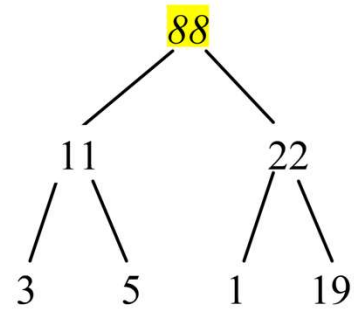
Append 88, swap with 19, then swap with 22.



(a) Add 88 to a heap



(b) After swapping 88 with 19



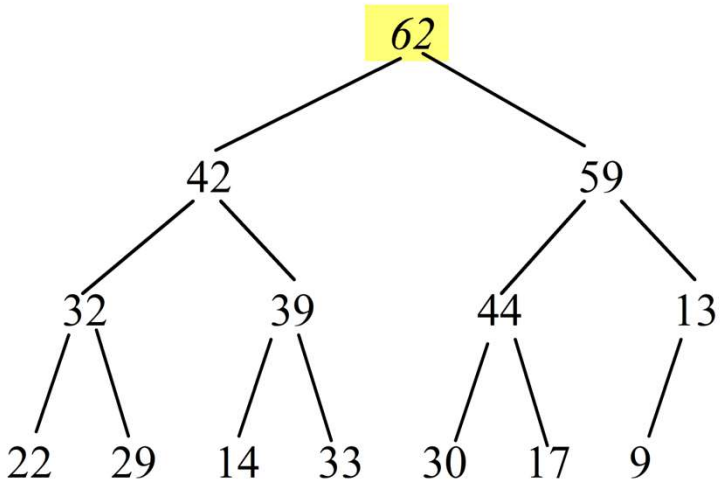
(b) After swapping 88 with 22

Two swaps in this example. At most the tree height in general.

Original diagram: © 2011 Pearson Addison-Wesley

Deletion example: remove 62

Save 62 as the return value. The last node contains 9.

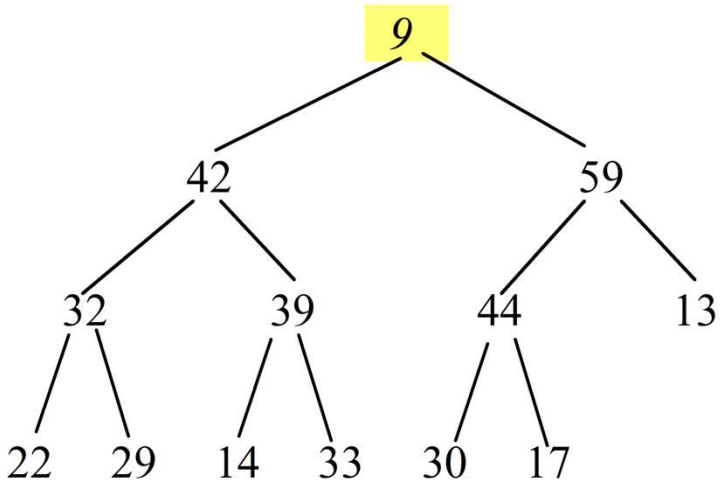


Finding the maximum and locating the last item each take O(1).

Original diagram: © 2011 Pearson Addison-Wesley

Deletion example: move 9 to the root

Remove the final node after moving its value.

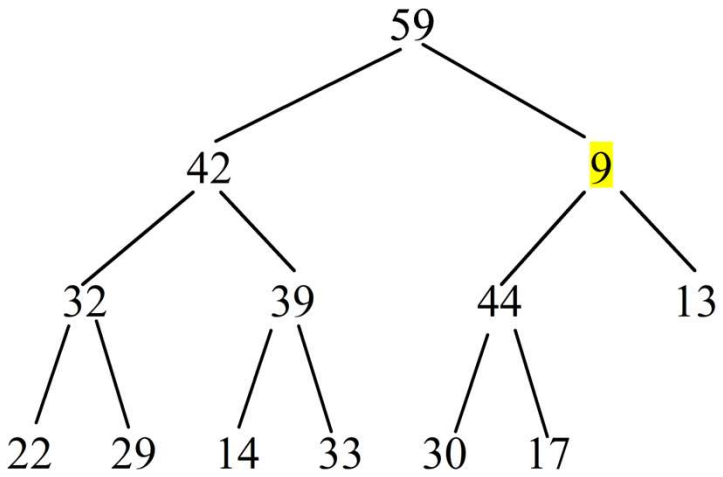


The shape is complete. The root violates max-heap order.

Original diagram: © 2011 Pearson Addison-Wesley

Deletion example: swap 9 with 59

59 is the larger of the root's children, 42 and 59.

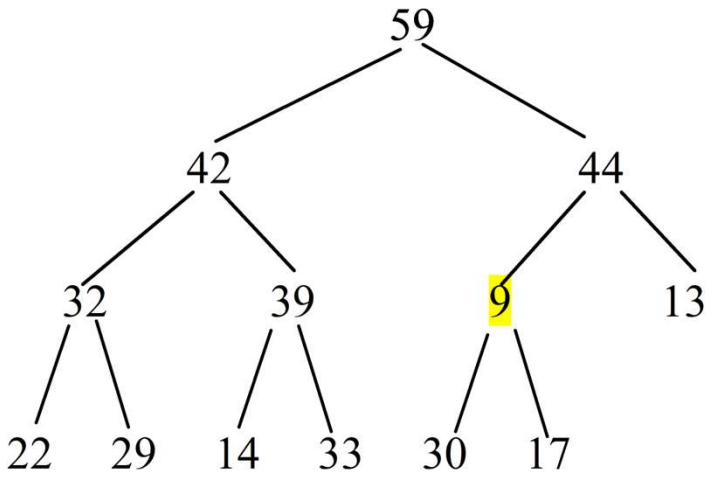


One sift-down step costs O(1). Continue from the new position of 9.

Original diagram: © 2011 Pearson Addison-Wesley

Deletion example: swap 9 with 44

44 is the larger of 9's children, 44 and 13.

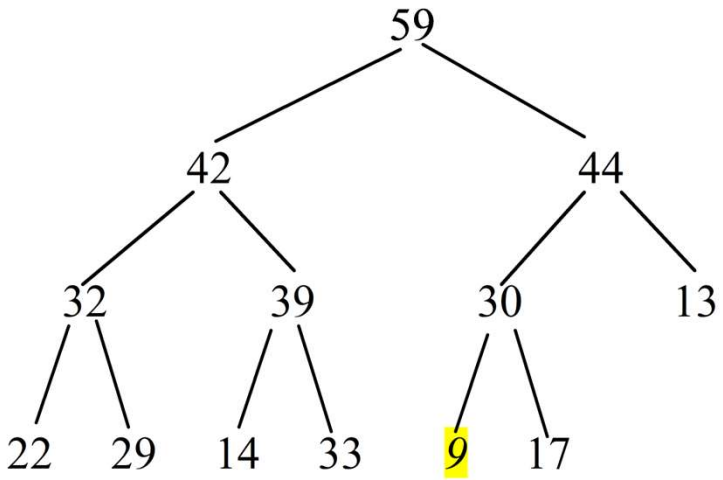


The possible violation moves down one level after each swap.

Original diagram: © 2011 Pearson Addison-Wesley

Deletion example: swap 9 with 30

30 is the larger child. After the swap, 9 reaches a leaf.



Three swaps restore the heap. General worst-case time: O(log n).

Original diagram: © 2011 Pearson Addison-Wesley

Building a heap from n items

Bottom-up construction is faster than inserting every item separately.

```
for (int i = list.size() / 2 - 1; i >= 0; i--)
    heapRebuild(i);
```

Leaves already satisfy heap order. Start at the last internal node.

Most nodes are near the leaves and move only a short distance.

Total work: n/4 + 2n/8 + 3n/16 + ... = O(n).

Bottom-up build: O(n). Repeated insertion: O(n log n) worst-case.

A heap-backed priority queue

Use the priority as the comparison key. Reverse comparisons for a min-heap.

Priority queue operation	Heap action	Time
add(item)	Append and sift up	O(log n)*
peek()	Read index 0	O(1)
remove()	Replace root, sift down	O(log n)
size() / isEmpty()	Read stored count	O(1)
clear()	Release n references	O(n)

*Amortized with geometric array growth. A resizing insertion can take O(n).

Heapsort

For ascending order, use a max-heap within the input array.

1. Build a max-heap in the array: O(n).

2. Swap the root with the last item in the active heap.

3. Shrink the active heap by one and sift down the new root.

4. Repeat for n − 1 removals: O(n log n) total.

Worst-case O(n log n). Iterative in-place heapsort uses O(1) extra space.

Heapsort and other comparison sorts

Properties depend on the implementation. These describe common array versions.

Algorithm	Worst-case time	Extra space / stability
Heapsort	O(n log n)	O(1) / unstable
Mergesort	O(n log n)	O(n) / stable
Quicksort	$O(n^2)$	Typically O(log n) stack / unstable

Quicksort has expected O(n log n) time with randomized pivots.

A heap array has local order. Removing all roots produces sorted order.

Java’s PriorityQueue

Natural ordering puts the smallest element at the head. A comparator can reverse it.

Methods	Behavior	Time
add(e), offer(e)	Insert an item	O(log n)*
poll(), remove()	Remove head	O(log n)
peek(), element()	Read head	O(1)
contains(o), remove(o)	Find an arbitrary item	O(n)
size(), isEmpty()	Read count	O(1)
clear()	Release references	O(n)

poll / peek return null when empty. remove() / element() throw.
*Heap work is O(log n). Array growth can make one insertion O(n).